

SIMATS ENGINEERING

ASSESSMENT TOOL 3

DSA0202 - COMPUTER VISION WITH OPENCV

Name : NAVADEEP R

Reg No : 192424110

Faculty Incharge : Mayuranathan M

CO2 – ASSESSMENT TOOL-3

Class Test

Code of Conduct:

I Navadeep R (192424110) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Navadeep R

Question 1: Image Preprocessing and Feature Extraction

Scenario: A drone is used to capture aerial images of a farmland to identify pest-affected crop regions. Due to sensor limitations, wind movement, and changing sunlight, the captured images contain noise, shadows, and uneven illumination. Pest-affected areas usually show visible texture and color variations compared to healthy crop regions.

Answer the following questions:

a) Preprocessing Pipeline — 4 Marks

Propose a suitable preprocessing pipeline to improve the quality of the farmland

images before analysis. Your answer should include methods for noise reduction,

illumination correction, contrast enhancement, and normalization.

Justify the purpose

of each step.

b) Feature Extraction — 3 Marks

Choose an appropriate feature extraction method to identify texture changes caused

by pest damage. Explain why the selected method is suitable for detecting affected

crop regions.

c) Rotation and Scale Variations — 3 Marks

Drone images may be captured from different heights and angles. Discuss how rotation and scale variations can affect feature detection. Suggest suitable methods to make the feature extraction process more robust.

Scenario

A drone captures aerial farmland images to identify **pest-affected crop regions**. Due to sensor limitations, wind movement, and changing sunlight, the images contain **noise, shadows, uneven illumination, and variations in texture and color**.

a) Preprocessing Pipeline – 4 Marks

A suitable preprocessing pipeline is:

Drone Image Acquisition



Noise Reduction



Illumination Correction



Contrast Enhancement



Image Normalization



Feature Extraction



Pest-Affected Region Detection

1. Noise Reduction

A **Gaussian filter** can be used to reduce random sensor noise.

The Gaussian filter smooths small unwanted intensity variations while maintaining the general structure of crop regions.

Purpose:

- Removes sensor noise.
- Produces a smoother image.
- Prevents noise from being interpreted as crop texture.

2. Illumination Correction

Changing sunlight and shadows cause some parts of the farmland to appear brighter or darker than others.

A suitable method is **background/illumination estimation followed by correction.**

One approach is:

$$I_{corrected} = \frac{I}{B + \epsilon}$$

where:

- I = original image
- B = estimated illumination
- ε = small value to avoid division by zero

Purpose:

- Reduce the effect of shadows.
- Correct uneven sunlight.
- Make crop regions more comparable.

3. Contrast Enhancement

CLAHE (Contrast Limited Adaptive Histogram Equalization) can be applied.

CLAHE improves local contrast rather than treating the entire image as one region.

Purpose:

- Makes pest-damaged regions more visible.
- Enhances local texture differences.
- Improves visibility in dark/shadowed areas.

4. Image Normalization

The image intensity values can be normalized to a common range such as **0–1** or **0–255**.

For example:

$$I_{norm} = \frac{I - I_{min}}{I_{max} - I_{min}}$$

Purpose:

- Gives consistent intensity values.
- Reduces the effect of different exposure levels.
- Makes feature extraction more stable.

Final Pipeline

Noise Reduction → Illumination Correction → CLAHE → Normalization

This produces a cleaner and more consistent farmland image for further analysis.

b) Feature Extraction – 3 Marks

For detecting texture changes caused by pest damage, **Local Binary Pattern (LBP)** is a suitable feature extraction method.

Local Binary Pattern (LBP)

LBP describes the texture of a pixel by comparing it with its neighboring pixels.

For each neighboring pixel:

- If neighbor $\geq$ center pixel $\rightarrow 1$

- If neighbor < center pixel $\rightarrow 0$

The resulting binary pattern represents local texture.

For example:

Neighbor pixels

1 1 0

1 C 0

0 1 1

The binary pattern is converted into a numerical feature.

Why LBP is suitable

Healthy and pest-affected crops can have different textures.

For example:

Healthy crop:

Smooth / regular texture

Pest-affected crop:

Irregular / damaged / rough texture

LBP can capture these local texture differences.

Therefore:

LBP is suitable for identifying local texture variations caused by pest damage.

It is also computationally simple, making it suitable for processing many drone images.

c) Rotation and Scale Variations – 3 Marks

Drone images may be captured at different **heights and camera angles**.

Rotation Variation

When the drone/camera changes orientation, the same crop texture may appear rotated.

A normal feature descriptor may produce different feature values for the same region.

Solution

Use **rotation-invariant LBP** or rotation-invariant feature descriptors.

SIFT and ORB can also provide rotation-aware features.

Scale Variation

If the drone flies higher, the crops appear smaller.

If the drone flies lower, the same crops appear larger.

Therefore, the same feature can occur at different scales.

Solution

Use **scale-invariant feature extraction methods** such as:

- SIFT

- SURF
- Multi-scale feature extraction

SIFT is particularly useful because it detects features at different scales.

Summary

Variation	Problem	Suitable Solution
Rotation	Feature orientation changes	Rotation-invariant LBP/SIFT/ORB
Scale	Feature size changes	SIFT/SURF/multi-scale features
Different camera angles	Feature appearance changes	Robust local descriptors

Thus, rotation- and scale-invariant features make the pest detection system more reliable.

Question 2: Stereo Vision and Motion Estimation

Scenario: A self-driving car is equipped with stereo cameras to navigate through a busy street. The system must estimate the distance of nearby vehicles and pedestrians in real time. It should also track moving objects accurately to avoid collisions, especially when pedestrians or vehicles change speed suddenly.

Answer the following questions:

a) Stereo Disparity and Depth Estimation — 4 Marks

Explain how stereo cameras use the difference between left and right image views to estimate the depth of objects. Discuss how disparity changes for nearby and distant objects.

b) Optical Flow for Motion Tracking — 3 Marks

If a pedestrian suddenly changes speed or direction, explain how optical flow helps in tracking the pedestrian's motion between consecutive video frames.

c) Errors and Mitigation Techniques — 3 Marks

Identify possible sources of error in depth estimation and motion tracking, such as camera misalignment, poor lighting, occlusion, fast motion, or textureless regions.

Suggest suitable techniques to reduce these errors.

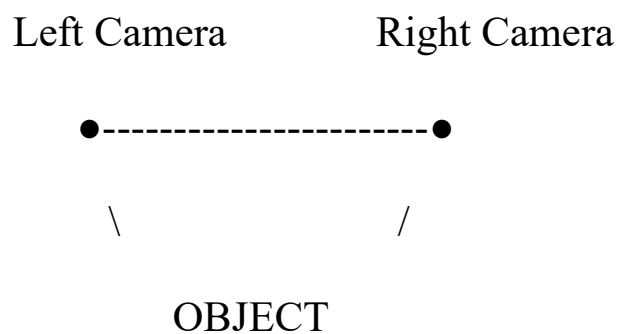
Scenario

A self-driving car uses **stereo cameras** to estimate the distance of vehicles and pedestrians and track their motion in real time.

a) Stereo Disparity and Depth Estimation – 4 Marks

Stereo vision uses **two cameras**, similar to human eyes.

The cameras capture the same scene from slightly different positions.



The same object appears at slightly different horizontal positions in the left and right images.

This difference is called **disparity**.

$$d = x_L - x_R$$

where:

- x_L = object position in left image
- x_R = object position in right image
- d = disparity

Depth is calculated using:

$$Z = \frac{fB}{d}$$

where:

- Z = depth/distance
- f = camera focal length
- B = distance between the two cameras (baseline)
- d = disparity

Nearby Objects

Nearby objects produce a **large disparity**.

d is large $\Rightarrow Z$ is small

Therefore:

Large disparity $\rightarrow$ Near object

Distant Objects

Distant objects produce a **small disparity**.

d is small $\Rightarrow Z$ is large

Therefore:

Small disparity → Far object

Example

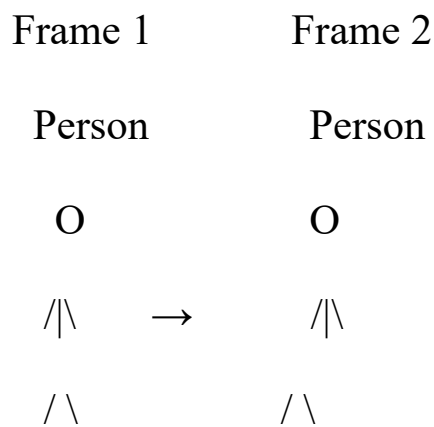
If a pedestrian is close to the car, their position will differ significantly between the left and right images.

If a building is far away, its position will differ only slightly.

b) Optical Flow for Motion Tracking – 3 Marks

Optical flow estimates the movement of pixels between consecutive video frames.

Suppose a pedestrian is moving:



The system compares the two frames and calculates motion vectors.

A motion vector contains:

- Direction
- Magnitude

The optical flow constraint equation is:

$$I_x u + I_y v + I_t = 0$$

where:

- u = horizontal movement
- v = vertical movement

Sudden speed change

If the pedestrian suddenly moves faster, the optical flow vectors become larger.

Normal:

→ → →

Sudden speed increase:

→→ →→ →→

Sudden direction change

If the pedestrian suddenly turns:

Before:

→ → →

After:

↗ ↗ ↗

the direction of the optical flow vectors changes.

A tracking system can use these changes to update the pedestrian's position and movement direction.

A **Kalman filter** can further improve tracking by predicting the pedestrian's next position.

c) Errors and Mitigation Techniques – 3 Marks

Several factors can cause errors in stereo depth estimation and optical-flow tracking.

Error	Effect	Mitigation
Camera misalignment	Incorrect disparity and depth	Stereo calibration and rectification
Poor lighting	Weak or unreliable features	Exposure correction, illumination normalization
Occlusion	Object becomes partially hidden	Object tracking + prediction
Fast motion	Motion blur and incorrect correspondence	Higher frame rate, robust optical flow
Textureless regions	Difficult to find corresponding points	Feature-based methods / additional sensors

1. Camera Misalignment

If the stereo cameras are not properly aligned, corresponding points may not be correctly matched.

Solution:

Perform **stereo camera calibration and image rectification**.

2. Poor Lighting

Dark areas, strong sunlight, and shadows make feature detection difficult.

Solution:

Use:

- Illumination correction
- Contrast enhancement
- HDR/exposure control

3. Occlusion

A pedestrian or vehicle may be partially hidden by another object.

This makes correspondence and tracking difficult.

Solution:

Use **object detection + Kalman filtering + multi-object tracking**.

The tracker can predict the object's position even when it is temporarily hidden.

4. Fast Motion

A rapidly moving pedestrian or vehicle may produce motion blur.

This can cause incorrect optical-flow vectors and stereo matching errors.

Solution:

Use:

- Higher camera frame rate
- Shorter exposure time
- Robust optical-flow algorithms
- Motion prediction

5. Textureless Regions

Plain walls, roads, or clothing may contain very few distinctive features.

Stereo matching becomes difficult because many pixels look similar.

Solution:

Use feature-based matching and combine stereo vision with additional sensors such as **LiDAR or radar** in practical autonomous vehicles.